

SMART LOST & FOUND MANAGEMENT SYSTEM

Project Report

Created By: SAHIL KIRAN WADHAI

Registration Number: 25BAI11392

Program: B.Tech

College: VIT Bhopal University

Course: Programming in Java

1. COVER PAGE

PROJECT REPORT

SMART LOST & FOUND MANAGEMENT SYSTEM

Created By:

SAHIL KIRAN WADHAI

Registration Number:

25BAI11392

Course:

Programming in Java

Program:

B.Tech

College:

VIT Bhopal University

Date of Submission:

18 September 2026

2. INTRODUCTION

The **Smart Lost & Found Management System** is a web-based application designed to manage lost and found items within a college campus.

Students frequently lose personal belongings such as mobile phones, wallets, watches, calculators, identity cards, books, bags, and other valuable items. In a traditional system, students usually depend on notice boards, social media groups, or informal communication to locate their belongings.

The proposed system provides a **centralized digital platform** where users can report lost and found items, search available records, identify possible matches, and submit claims.

The system also includes a **Smart Matching Engine** that compares lost and found items based on several characteristics such as:

- Category
- Item name
- Location
- Color
- Brand
- Date
- Description

The project uses **Java, Java Servlets, JDBC, MySQL, Maven, HTML, CSS, JavaScript/React and TypeScript** to implement the application.

An administrator module is also provided to manage users, items, claims, and matching records.

3. PROBLEM STATEMENT

Managing lost and found items manually in a college campus can be difficult and inefficient.

When a student loses an item, there may be no centralized place where the student can register the lost item. Similarly, when another student finds an item, it may not be possible to identify its owner.

The major problems with the traditional approach are:

- No centralized database of lost and found items.
- Difficult to search through large numbers of reports.

- Lost and found information may be scattered across different platforms.
- Manual comparison of item descriptions is time-consuming.
- Students may not know when their lost item has been found.
- Ownership verification can be difficult.
- Administrators have difficulty maintaining records.

The **Smart Lost & Found Management System** solves these problems by providing a centralized platform with database storage and an automated matching mechanism.

4. FUNCTIONAL REQUIREMENTS

Functional requirements describe what the system should be able to do.

4.1 User Registration

The system allows students to create an account by providing information such as:

- Name
 - Student ID
 - Email
 - Phone number
 - Password
-

4.2 User Login

Registered users can log in to the system using their credentials.

The system authenticates the user and provides access to the appropriate dashboard.

4.3 Report Lost Item

Students can report an item they have lost.

The report contains:

- Item name

- Category
 - Color
 - Brand
 - Description
 - Location
 - Date
 - Image
 - Status
-

4.4 Report Found Item

Users can report an item they have found on campus.

The same type of descriptive information can be provided to help identify its owner.

4.5 Search Items

Users can search for lost or found items based on available information.

Search criteria may include:

- Item name
 - Category
 - Location
 - Color
 - Brand
 - Item type
-

4.6 Smart Matching

The system automatically compares lost and found records.

The matching engine uses seven criteria:

Criterion	Weight
-----------	--------

Category	20%
----------	-----

Item Name	20%
Location	20%
Color	10%
Brand	10%
Date	10%
Description	10%
Total	100%

4.7 Match Results

The system displays:

- Matching item
- Match score
- Match category
- Matched fields

The categories are:

- **High Match:** 80–100%
 - **Medium Match:** 60–79%
 - **Low Match:** 40–59%
 - **Unlikely Match:** Below 40%
-

4.8 Claim Submission

A student can submit a claim if they believe a found item belongs to them.

The claim contains a verification message and claim status.

4.9 Claim Verification

The administrator reviews claims and can:

- Approve the claim
 - Reject the claim
-

4.10 Administrator Management

The administrator can manage:

- Users
 - Items
 - Matches
 - Claims
 - Item statuses
-

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Usability

The system should have a simple and user-friendly interface.

5.2 Performance

Searching and matching operations should execute efficiently.

5.3 Reliability

Invalid operations should be handled properly without crashing the system.

5.4 Security

User authentication and password protection are provided.

The project includes password hashing using SHA-256 with a salt.

5.5 Maintainability

The Java backend is separated into different packages and layers, making it easier to modify and maintain.

5.6 Scalability

MySQL database storage provides a foundation for handling a larger number of users and item records.

5.7 Modularity

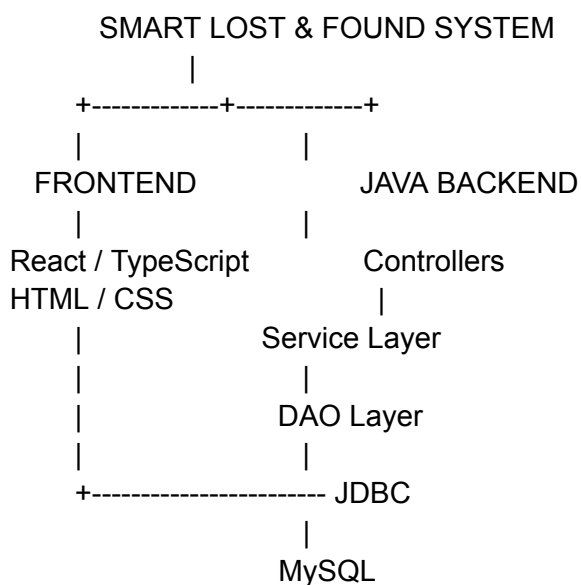
The system separates:

- Model
- DAO
- Service
- Controller
- Utility

components.

6. SYSTEM ARCHITECTURE

The system follows a layered architecture.



6.1 Frontend Layer

Responsible for:

- User interface
- Forms
- Navigation
- Dashboards
- Search
- Match display
- Claims

6.2 Controller Layer

Java Servlets receive and process HTTP requests.

Examples include:

- LoginServlet
- RegisterServlet
- LostItemServlet
- FoundItemServlet
- SearchServlet
- MatchServlet
- ClaimServlet
- AdminServlet

6.3 Service Layer

Contains business logic.

Important services include:

- LoginService
- UserService
- ItemService
- MatchingService
- ClaimService

6.4 DAO Layer

Responsible for database operations.

Examples:

- UserDAO
- ItemDAO
- MatchDAO
- ClaimDAO

6.5 Model Layer

Represents the main entities:

- User
- Student
- Admin
- Item
- LostItem
- FoundItem
- Match
- Claim

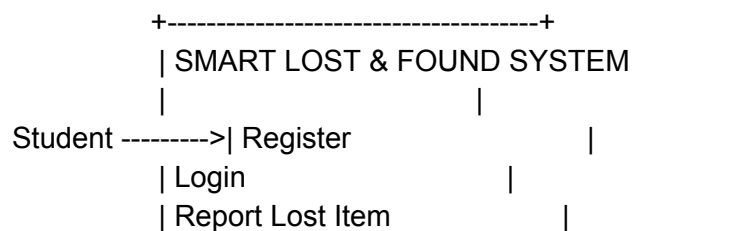
6.6 Database Layer

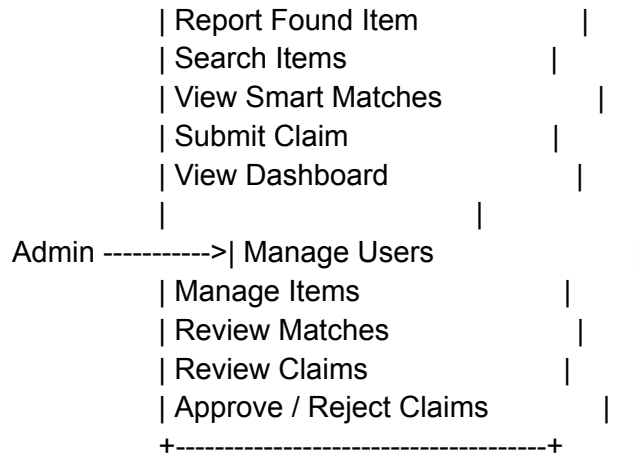
MySQL stores:

- Users
 - Administrators
 - Items
 - Matches
 - Claims
-

7. DESIGN DIAGRAMS

7.1 USE CASE DIAGRAM





Actors

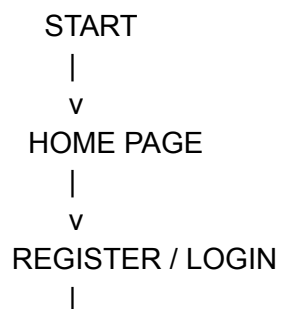
Student/User

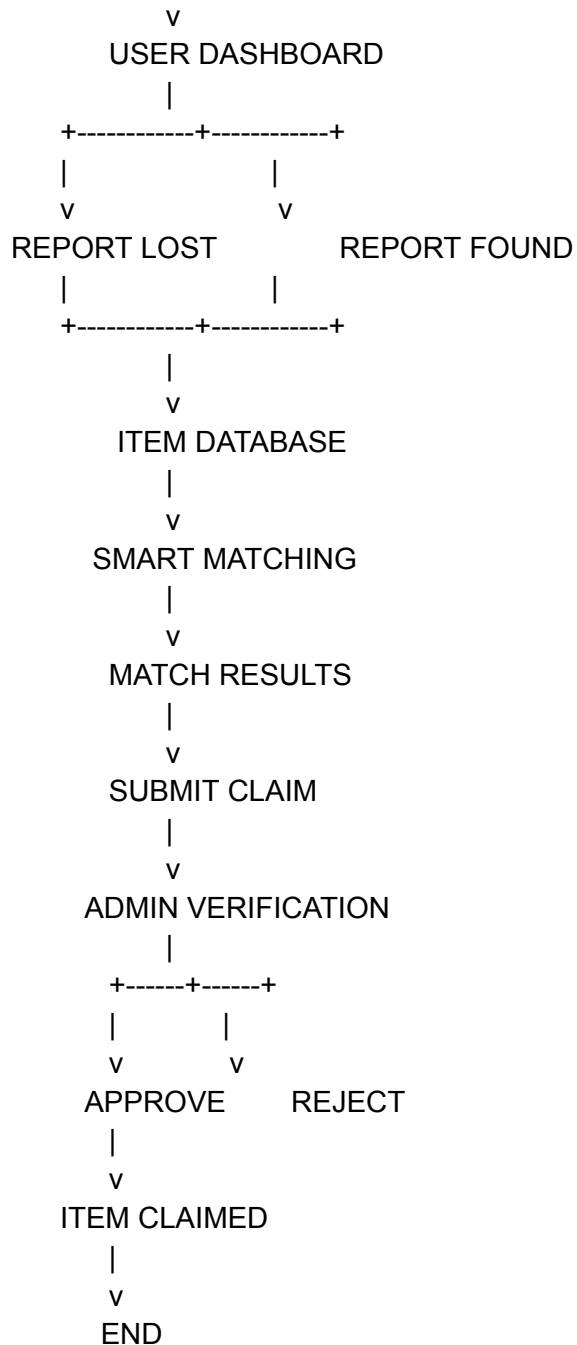
- Register
- Login
- Report lost item
- Report found item
- Search
- View matches
- Submit claims

Administrator

- Manage users
- Manage items
- Review claims
- Manage matches

7.2 WORKFLOW DIAGRAM

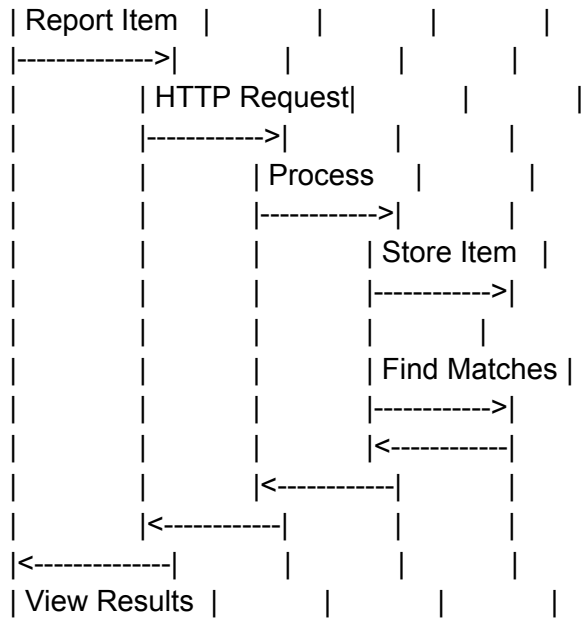




7.3 SEQUENCE DIAGRAM

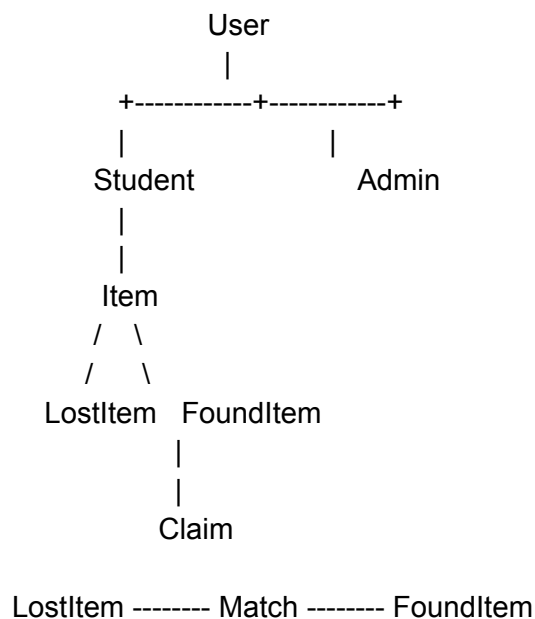
Example: Student reports a lost item and the system searches for matches.

Student	Frontend	Servlet	Service	Database

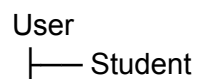


7.4 CLASS / COMPONENT DIAGRAM

Class Diagram



Important Classes



└─ Admin

Item

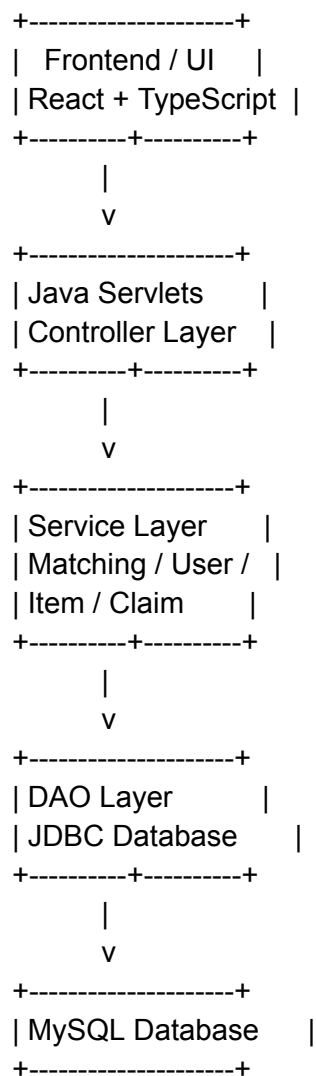
└─ LostItem

└─ FoundItem

Match

Claim

Component Diagram



7.5 ER DIAGRAM

```

+-----+
|  USERS  |
+-----+
| user_id (PK) |
| name        |
| student_id  |
| email       |
| phone       |
| password    |
| role        |
+-----+

```

```

|
| 1
|
| M
v

```

```

+-----+
|  ITEMS  |
+-----+
| item_id (PK) |
| user_id (FK) |
| item_type    |
| item_name    |
| category     |
| color        |
| brand        |
| description   |
| location     |
| item_date    |
| status       |
+-----+

```

```

|
|
+-----+
|           |
|           |
v           v

```

```

+-----+ +-----+
| MATCHES | | CLAIMS |
+-----+ +-----+
| match_id (PK) | | claim_id (PK) |
| lost_item_id | | item_id (FK) |
| found_item_id | | claimant_id |
| match_score  | | message   |

```

```
| match_status | | status |  
+-----+ +-----+
```

8. DESIGN DECISIONS & RATIONALE

8.1 Layered Architecture

A layered architecture was selected to separate different responsibilities.

This makes the application easier to understand, test, and maintain.

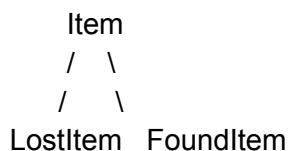
8.2 Object-Oriented Design

Java classes represent real-world entities such as users, items, matches, and claims.

This makes the system easier to extend.

8.3 Inheritance

`LostItem` and `FoundItem` inherit common properties from `Item`.



This reduces duplicate code.

8.4 Weighted Matching

The matching algorithm uses seven criteria instead of depending on only one attribute.

This allows the system to consider multiple characteristics of an item.

8.5 MySQL Database

MySQL was selected because the project contains structured relationships between:

- Users
- Items
- Matches
- Claims

8.6 DAO Pattern

DAO classes separate database operations from business logic.

This improves code organization and maintainability.

9. IMPLEMENTATION DETAILS

9.1 Model Package

The model package contains:

User.java
Student.java
Admin.java
Item.java
LostItem.java
FoundItem.java
Match.java
Claim.java

9.2 DAO Package

Contains:

UserDAO.java
ItemDAO.java
MatchDAO.java
ClaimDAO.java

These classes communicate with the MySQL database through JDBC.

9.3 Service Package

Contains:

MatchingService.java
LoginService.java
UserService.java
ItemService.java
ClaimService.java

The `MatchingService` implements the main matching algorithm.

9.4 Controller Package

Contains Java Servlets such as:

LoginServlet.java
RegisterServlet.java
LostItemServlet.java
FoundItemServlet.java
SearchServlet.java
MatchServlet.java
ClaimServlet.java
AdminServlet.java

9.5 Utility Package

Contains utility classes such as:

DatabaseConnection.java
PasswordUtil.java

9.6 Maven

The backend uses Maven for:

- Dependency management
- Compilation
- Packaging
- Build management

The backend can be packaged as a WAR file and deployed on Apache Tomcat.

10. SCREENSHOTS / RESULTS

For the final report, this section should contain screenshots from your actual project.

Recommended screenshots:

Screenshot 1 – Home Page

Shows the main landing page of the application.

Caption:

Figure 1: Smart Lost & Found System Home Page

Screenshot 2 – Registration Page

Shows the student registration form.

Caption:

Figure 2: User Registration Page

Screenshot 3 – Login Page

Shows the user authentication interface.

Caption:

Figure 3: Login Page

Screenshot 4 – Student Dashboard

Shows reported items, matches, and claims.

Caption:

Figure 4: Student Dashboard

Screenshot 5 – Report Lost Item

Shows the form used to report a lost item.

Caption:

Figure 5: Report Lost Item Page

Screenshot 6 – Report Found Item

Shows the form used to report a found item.

Caption:

Figure 6: Report Found Item Page

Screenshot 7 – Search Page

Shows item search and filtering.

Caption:

Figure 7: Search Items Page

Screenshot 8 – Smart Match Results

Shows:

- Match score
- Match category
- Matched fields

Caption:

Figure 8: Smart Matching Results

Screenshot 9 – Claims Page

Shows submitted ownership claims.

Caption:

Figure 9: Claims Management Page

Screenshot 10 – Admin Dashboard

Shows administrator functions.

Caption:

Figure 10: Administrator Dashboard

11. TESTING APPROACH

Testing was performed at the application and Java levels.

11.1 Functional Testing

Test Case	Input	Expected Result
Registration	Valid details	Account created
Login	Valid credentials	Login successful
Lost Item	Valid details	Item recorded
Found Item	Valid details	Item recorded
Search	Item keyword	Relevant records displayed
Matching	Lost + Found records	Match score calculated
Claim	Found item	Claim created
Approve Claim	Valid claim	Claim approved
Reject Claim	Invalid claim	Claim rejected

11.2 Matching Algorithm Testing

The matching system can be tested with different combinations of:

- Same category
- Different category

- Same location
 - Different location
 - Same brand
 - Different brand
 - Same color
 - Different color
 - Same date
 - Different dates
 - Similar descriptions
-

11.3 Java Testing

Java implementation can be tested for:

- Object creation
 - Constructors
 - Getters and setters
 - Inheritance
 - Method overriding
 - Polymorphism
 - Collections
 - Matching calculations
 - Exception handling
 - Database operations
-

12. CHALLENGES FACED

The major challenges involved in developing the project include:

1. Designing a centralized lost-and-found system.
2. Managing both lost and found item records.
3. Designing the database relationships.
4. Developing the seven-criterion matching algorithm.
5. Implementing text similarity.
6. Handling date-based matching.
7. Connecting Java with MySQL using JDBC.
8. Maintaining communication between frontend and backend.
9. Managing user authentication.
10. Handling claims and verification.

11. Organizing Java classes into appropriate packages.
 12. Maintaining consistency between frontend and backend matching logic.
-

13. LEARNINGS & KEY TAKEAWAYS

This project helped demonstrate practical knowledge of:

- Java programming
- Object-Oriented Programming
- Classes and Objects
- Encapsulation
- Inheritance
- Polymorphism
- Method Overriding
- Exception Handling
- Java Servlets
- JDBC
- MySQL
- Maven
- Database Design
- Frontend Development
- Backend Development
- REST-style communication
- Matching Algorithms
- Software Architecture

The project also demonstrates how theoretical programming concepts can be combined to develop a practical real-world application.

14. FUTURE ENHANCEMENTS

The following features can be added in future versions.

14.1 Image-Based Matching

Computer vision could be used to compare uploaded photographs of lost and found objects.

14.2 Notifications

The system could send:

- Email notifications
- SMS notifications
- Push notifications

when a possible match is found.

14.3 Mobile Application

An Android/iOS application could provide easier access to the system.

14.4 QR Code Integration

QR codes could be used to identify registered belongings.

14.5 Advanced Authentication

Additional roles could be introduced:

- Student
- Faculty
- Security Staff
- Administrator
- Campus Authorities

14.6 Advanced Analytics

The system could generate statistics about:

- Lost items
- Found items
- Recovered items
- Pending claims
- Frequently reported locations

14.7 Cloud Deployment

The application could be deployed to a cloud server with a managed MySQL database.

14.8 Improved Password Security

For production use, stronger password-hashing methods such as Argon2 or bcrypt could be considered.

15. REFERENCES

1. Oracle Java Documentation
2. Java Platform Documentation
3. MySQL Documentation
4. Apache Tomcat Documentation
5. Maven Documentation
6. Java Servlet API Documentation
7. JDBC Documentation
8. React Documentation
9. TypeScript Documentation
10. Gson Documentation